

Abstraktion

Kriterium	Beschreibung	Wann zu verwenden?
Definition von Abstraktion	Abstraktion bedeutet, dass nur die wesentlichen Eigenschaften und Verhaltensweisen eines Objekts dargestellt werden, während unwichtige Details verborgen bleiben.	Wenn eine Klasse oder ein Objekt nur auf eine bestimmte Weise verwendet werden soll, ohne sich mit den Details der Implementierung zu befassen.
Verwendung von abstrakten Klassen	Eine abstrakte Klasse ist eine Klasse, die nicht instanziiert werden kann und als Basis für andere Klassen dient. Sie kann sowohl abstrakte als auch konkrete Methoden enthalten.	Wenn eine allgemeine Klasse bereitgestellt werden soll, die von anderen Klassen erweitert werden muss, aber keine vollständige Implementierung bietet.
Verwendung von Schnittstellen	Eine Schnittstelle definiert nur die Signaturen von Methoden, ohne eine konkrete Implementierung zu bieten. Sie dient als Vertrag für die Klassen, die sie implementieren.	Wenn eine Gruppe von Klassen dasselbe Verhalten aufweisen soll, aber unterschiedliche Implementierungen benötigen.
Ziel der Abstraktion	Abstraktion zielt darauf ab, die Komplexität zu verringern, indem nur die relevanten Details präsentiert werden. Dies ermöglicht eine klare Trennung zwischen den "Was" und den "Wie".	Wenn die Implementierungsdetails nicht von der restlichen Software gesehen werden sollen, sondern nur die Interaktion mit einem Objekt von Interesse ist.
Verhinderung von Überinformation	Abstraktion hilft, den Code übersichtlicher und verständlicher zu machen, indem sie nur die wesentlichen Eigenschaften und Methoden zeigt.	Wenn unnötige Details vom Benutzer oder anderen Klassen verborgen werden sollen, um die Nutzung zu vereinfachen.
Förderung der Wiederverwendbarkeit	Abstraktion ermöglicht es, allgemeine Funktionalitäten in abstrakten Klassen oder Schnittstellen zu definieren, die von verschiedenen konkreten Implementierungen genutzt werden können.	Wenn dieselbe Logik oder Funktionalität in verschiedenen Klassen benötigt wird, aber die Implementierung unterschiedlich sein soll.

Kriterium	Beschreibung	Wann zu verwenden?
Unterschied zu Kapselung	Während Kapselung dazu dient, Details einer Klasse zu verbergen und den Zugriff zu steuern, geht es bei der Abstraktion darum, den Fokus auf das Wesentliche zu lenken und unwichtige Details auszublenden.	Wenn der Fokus auf der Struktur und den gemeinsamen Funktionen von Objekten liegt, ohne sich mit den Details ihrer Implementierung zu befassen.
Vermeidung von engen Kopplungen	Durch Abstraktion können Klassen und Komponenten so gestaltet werden, dass sie nur über Schnittstellen oder abstrakte Klassen miteinander interagieren, anstatt direkt auf konkrete Implementierungen zuzugreifen.	Wenn Klassen und Komponenten unabhängig voneinander entwickelt und getestet werden sollen.
Erhöhung der Flexibilität	Abstraktion ermöglicht es, das Verhalten von Objekten zur Laufzeit zu ändern oder auszutauschen, ohne dass andere Teile des Systems beeinträchtigt werden.	Wenn ein System oder eine Komponente flexibel und leicht erweiterbar sein soll.
Verwendung von abstrakten Methoden	Abstrakte Methoden in abstrakten Klassen und Schnittstellen sind Methoden, die von den abgeleiteten Klassen implementiert werden müssen, aber keine eigene Implementierung in der abstrakten Klasse haben.	Wenn eine allgemeine Schnittstelle bereitgestellt werden soll, aber die konkrete Implementierung in den abgeleiteten Klassen erfolgt.
Zugriff auf konkrete Implementierungen	Abstraktion trennt die Darstellung der Funktionalität von ihrer konkreten Implementierung, sodass der Benutzer nur mit den abstrahierten Methoden interagiert, ohne die Implementierung zu kennen.	Wenn nur die Funktionalität und nicht die konkrete Implementierung eines Objekts benötigt wird, z. B. beim Arbeiten mit Schnittstellen.
Verwendung bei Schnittstellen	Schnittstellen bieten eine Form der Abstraktion, indem sie nur die Signaturen von Methoden definieren, die von der implementierenden Klasse bereitgestellt werden müssen.	Wenn ein Verhalten in verschiedenen Klassen definiert werden soll, aber die Implementierungen flexibel bleiben müssen.

Kriterium	Beschreibung	Wann zu verwenden?
Ermöglichung der Erweiterbarkeit	Abstraktion sorgt dafür, dass das System leicht erweiterbar bleibt, da neue konkrete Klassen hinzugefügt werden können, ohne bestehende Klassen zu verändern.	Wenn das System zukünftige Erweiterungen und neue Funktionalitäten unterstützen soll.
Abstraktion und Polymorphismus	Abstraktion in Verbindung mit Polymorphismus ermöglicht es, dass Objekte verschiedener konkreter Typen über eine gemeinsame Schnittstelle oder abstrakte Klasse referenziert und verwendet werden können.	Wenn Objekte verschiedener Klassen auf die gleiche Weise behandelt werden sollen, ohne ihre konkreten Typen zu kennen.

Zusammenfassung

Abstraktion ist ein grundlegendes Konzept in der objektorientierten Programmierung, das darauf abzielt, die Komplexität zu verringern, indem nur die wesentlichen Details einer Klasse oder eines Objekts sichtbar gemacht werden, während die weniger wichtigen Implementierungsdetails verborgen bleiben. Abstraktion wird durch abstrakte Klassen und Schnittstellen erreicht, die die Grundlage für verschiedene Implementierungen bieten und Flexibilität sowie Erweiterbarkeit ermöglichen.

Wann verwenden?

- Wenn die Implementierungsdetails eines Objekts nicht für den Benutzer sichtbar sein sollen, sondern nur die wesentlichen Eigenschaften und Verhaltensweisen.
- Wenn mehrere Klassen ein ähnliches Verhalten aufweisen sollen, jedoch unterschiedliche Implementierungen haben.
- Wenn eine gemeinsame Struktur oder Funktionalität bereitgestellt werden soll, die von verschiedenen konkreten Klassen implementiert werden kann.

Vorteile

- **Verringert Komplexität**
Nur die relevanten Details sind sichtbar, was das Verständnis und die Nutzung des Codes vereinfacht.
- **Erhöht Flexibilität und Erweiterbarkeit**
Neue Implementierungen können hinzugefügt werden, ohne bestehende Implementierungen zu verändern.
- **Fördert Wiederverwendbarkeit**
Abstrakte Klassen und Schnittstellen ermöglichen es, eine allgemeine Struktur zu definieren, die von mehreren Klassen verwendet werden kann.

Nachteile

- **Overhead**
Die Verwendung von abstrakten Klassen und Schnittstellen kann zusätzlichen Aufwand und Komplexität in der Implementierung verursachen.

- **Verborgene Implementierungsdetails**

Während Abstraktion eine nützliche Vereinfachung darstellt, kann sie auch dazu führen, dass der Entwickler die zugrunde liegende Implementierung nicht versteht.
